



CIBERSEGURIDAD EN ENTORNOS DE LAS TECNOLOGÍAS DE LA INFORMACIÓN

Puesta en Producción Segura

TEMA 3

Detección y corrección de vulnerabilidades de aplicaciones web
Alberto Diéguez Álvarez

Tabla de contenido

Caso práctico	3
Apartado 1: Inyección de Cross Site Scripting (XSS)	4
Apartado 2: Preguntas sobre el ejercicio	10

Caso práctico



[@lucabravo para unsplash](#). [computadora-portatil-gris-encendida](#) (Licencia [Unsplash](#))

Julián ha estado probando distintos tipos de vulnerabilidades y ataques que podría sufrir su aplicativo web, pero aún tiene pruebas y escenarios que testar.

Sus compañeros de trabajo le han comentado que vigile si su aplicativo web pudiera ser víctima de algún tipo de *Cross Site Scripting* (XSS).

Para ello, **Julián** revisará un formulario para introducir comentarios en un foro que le preocupa pueda ser víctima de algún tipo de XSS.

¿Qué te pedimos que hagas?

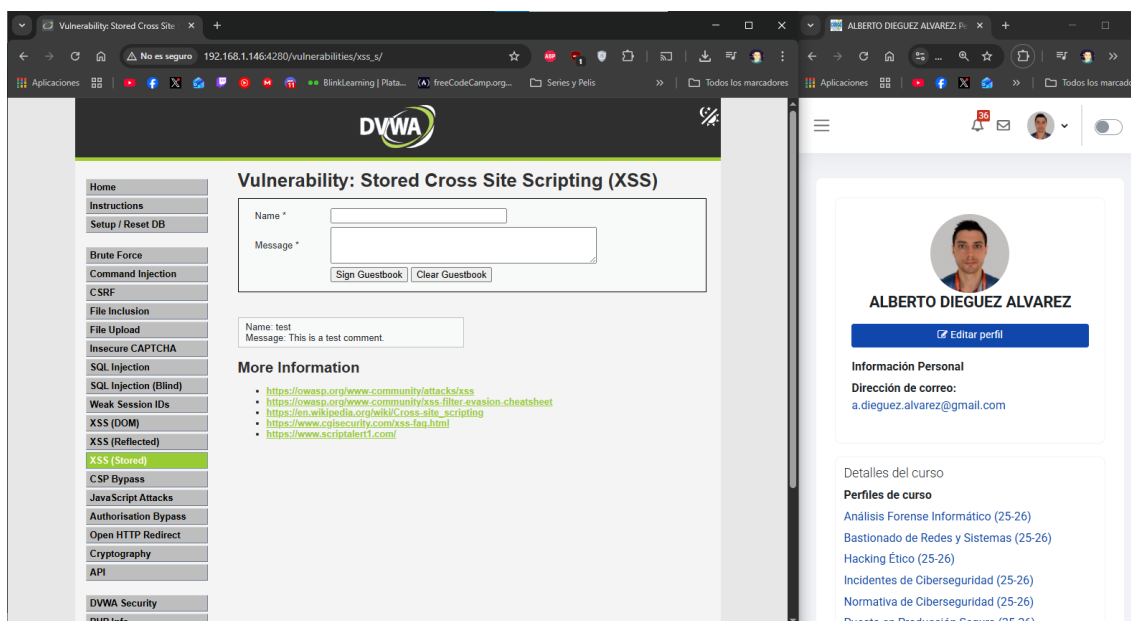
Apartado 1: Inyección de Cross Site Scripting (XSS)

1. Trabajaremos sobre el entorno de DVWA que construimos en la unidad anterior.
2. Arranca el aplicativo de DVWA (en la unidad 2 ya vimos como lanzarla) y accede en un navegador a <http://localhost:4280>

Como ya lo tenia de la unidad anterior, recreo el contenedor, pongo la VM en modo puente y abro el puerto, entonces puedo acceder desde Windows y con el usuario 'admin' y la contraseña 'password' accedo a la herramienta.

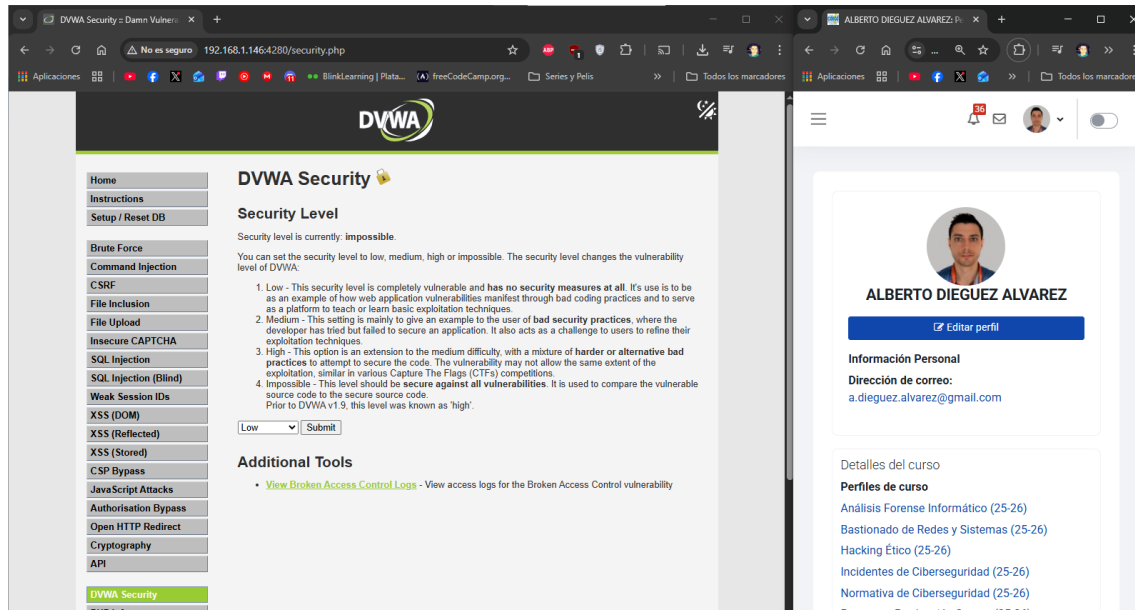
3. Nos desplazaremos hasta el módulo de XSS del menú lateral izquierdo. En este caso de tipo XSS almacenado (XSS Stored)

Accedo al apartado mencionado.



4. Seguimos trabajando en el modo Low (configurado en el menú de Security)

Compruebo el modo de seguridad y lo pongo en Low ya que estaba en imposible.



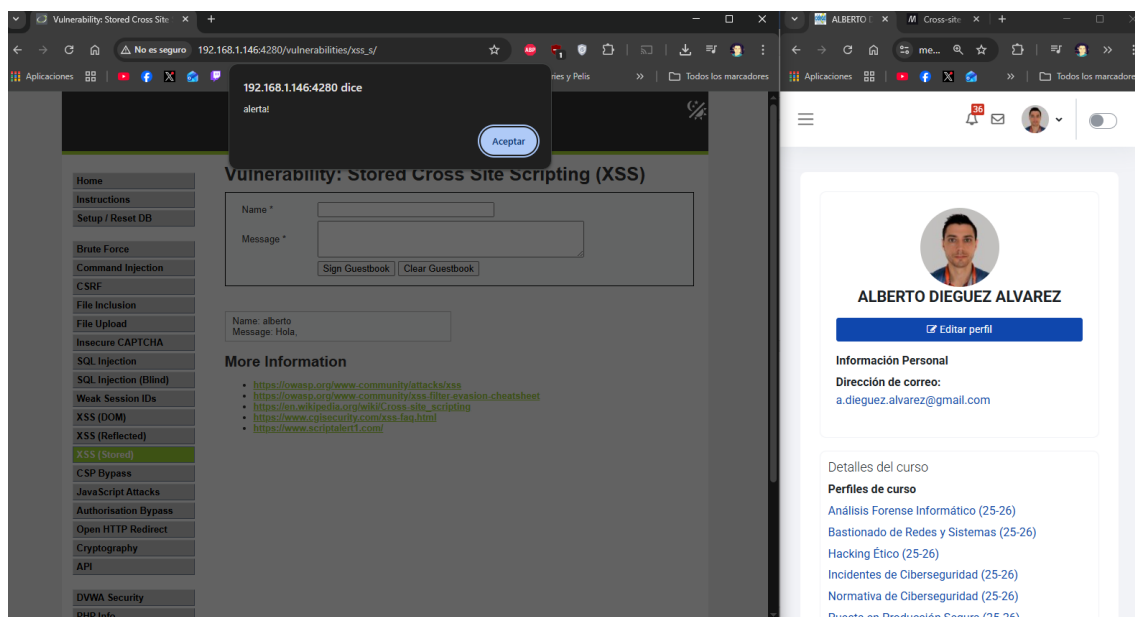
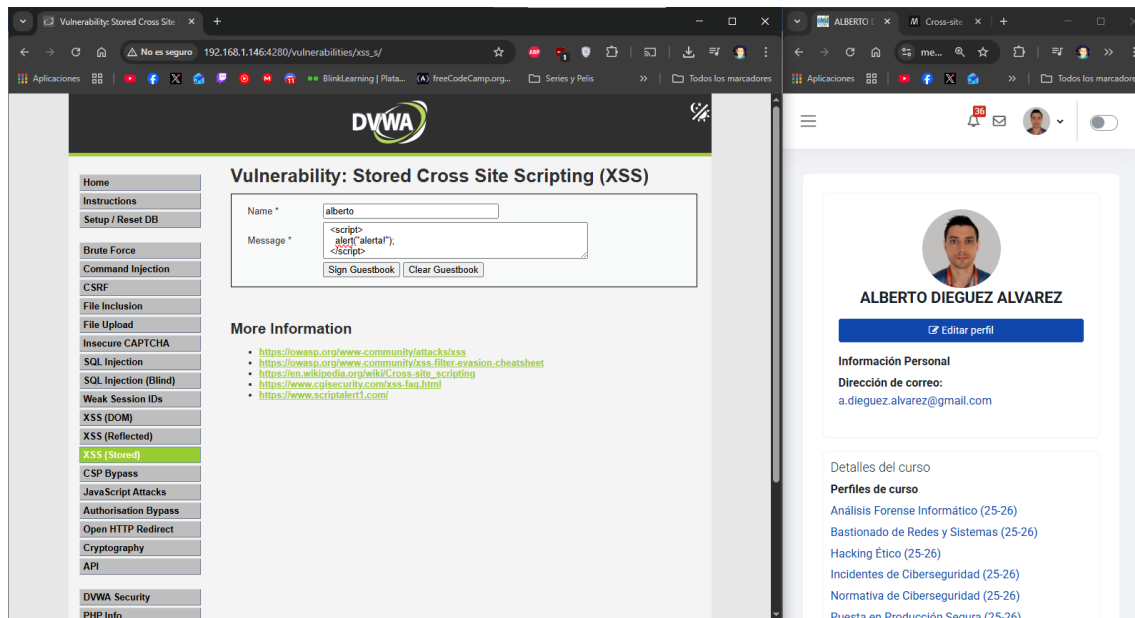
5. ¿Podrías conseguir que muestre una ventana de alerta con un mensaje cada vez que alguien visita el libro de firmas?

Ayuda: los navegadores interpretan Javascript. Si los mensajes que se graban en esta pantalla se muestran a todos los usuarios, y grabo como mensaje un código javascript, este se podría ejecutar en todos los clientes que abran esa página si la aplicación no está protegida.

Muestro el mensaje de alerta, introduciendo etiquetas script para que se ejecute el código en javascript.

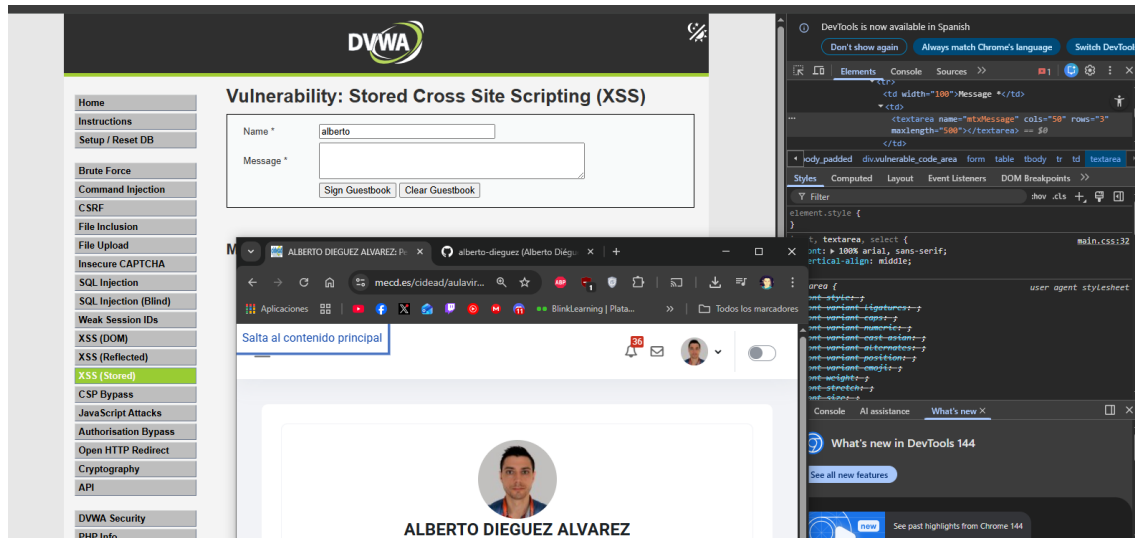
Código.

```
<script> alert("alerta"); </script>
```



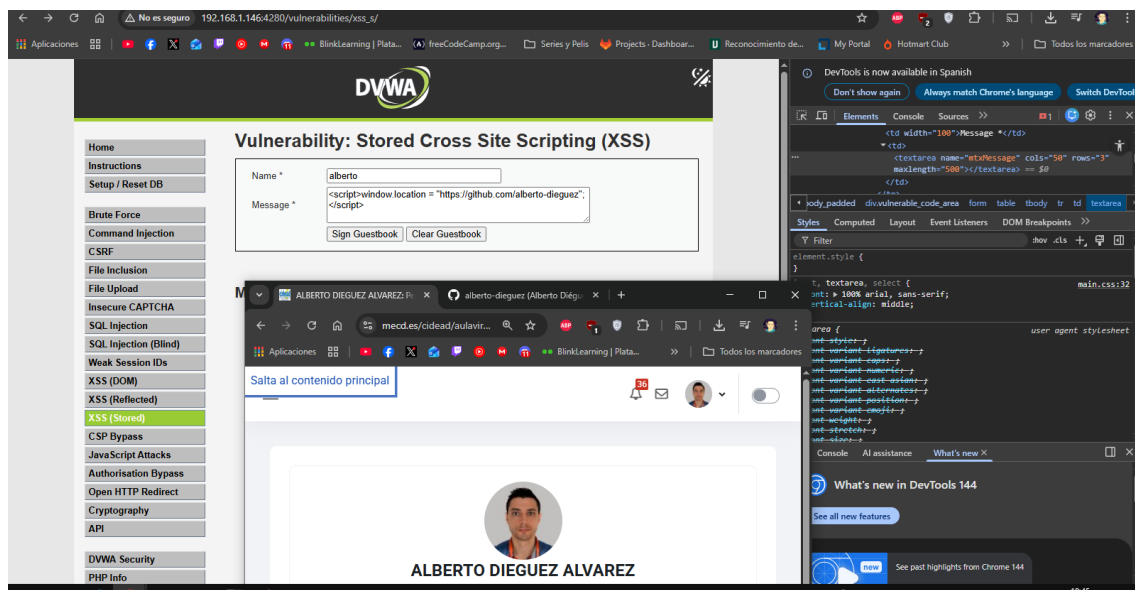
6. ¿Serías capaz de introducir un XSS que redirija al usuario a una web de tu elección cada vez que se visite el libro de firmas? (por ejemplo Youtube.com o Google.com)

Para hacer eso modifiko el max lenght del textarea del message para poder escribir todo el código que necesito.

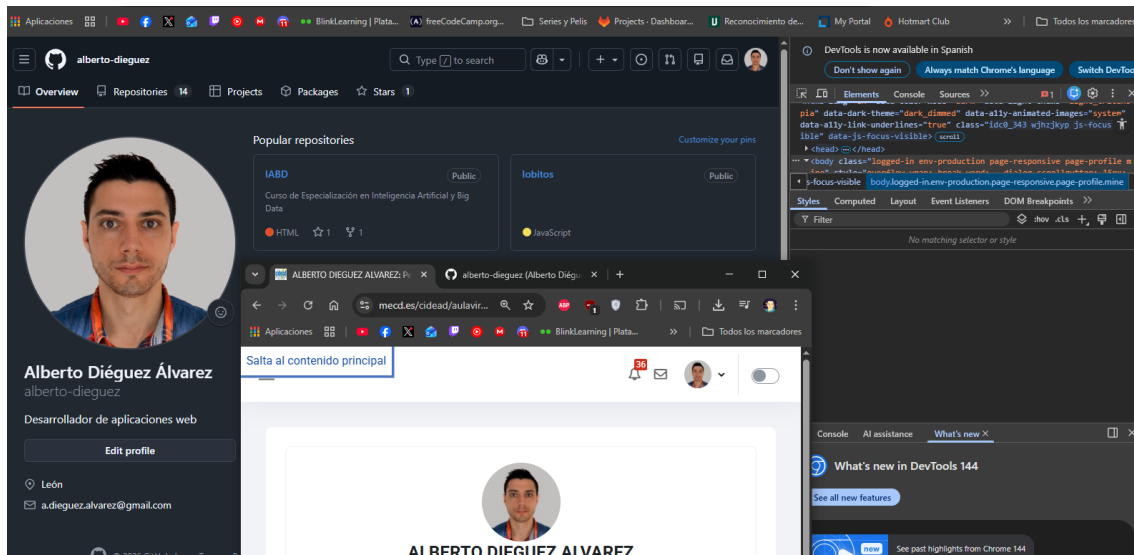


Ahora ingreso el payload.

`<script>window.location = "https://github.com/alberto-dieguez";</script>`



Según ejecuto el payload me redirecciona.



7. ¿Serías capaz de robar la cookie del usuario? ¿Por ejemplo redirigiéndola a un servidor tuyo?

Ayuda: puedes crear de forma sencilla un servidor web que escuche peticiones en tu máquina local mediante un contenedor (por ejemplo, mira https://hub.docker.com/_/nginx). Ejecuta la siguiente línea de comandos:

```
docker run --name nginx --rm -p 8080:80 nginx
```

Con esto podrás ver en la consola/terminal como recibe tu servidor la cookie

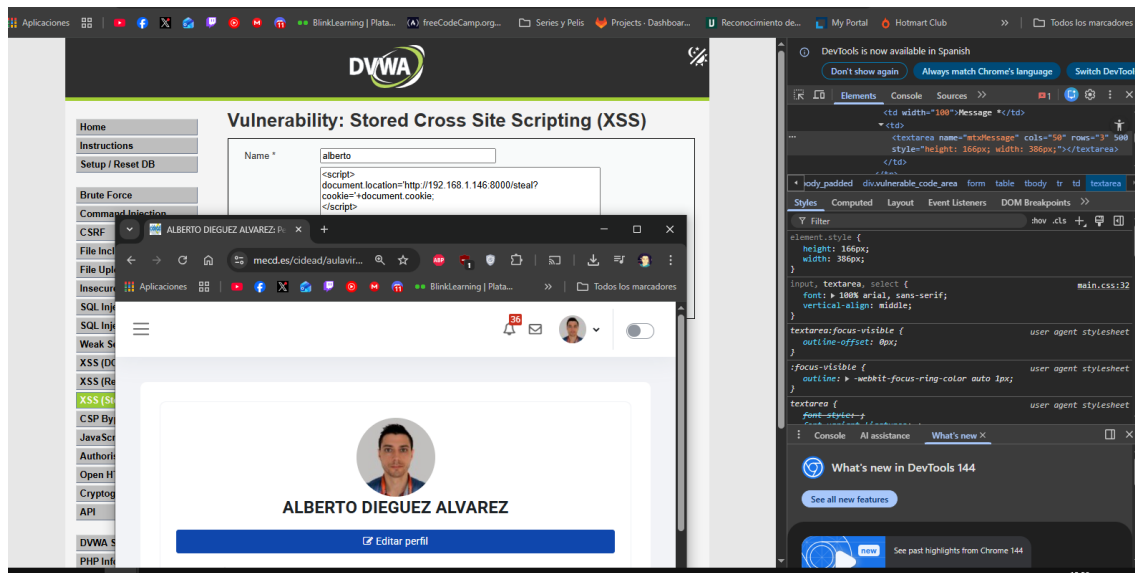
NOTA: para parar un contenedor lanzado en primero plano se puede simplemente cerrar la terminal o pulsar CTRL+C. Después para eliminar el contenedor (en el caso de que no se haya eliminado al cerrar la ventana) ejecuta:

```
docker rm -f nginx
```

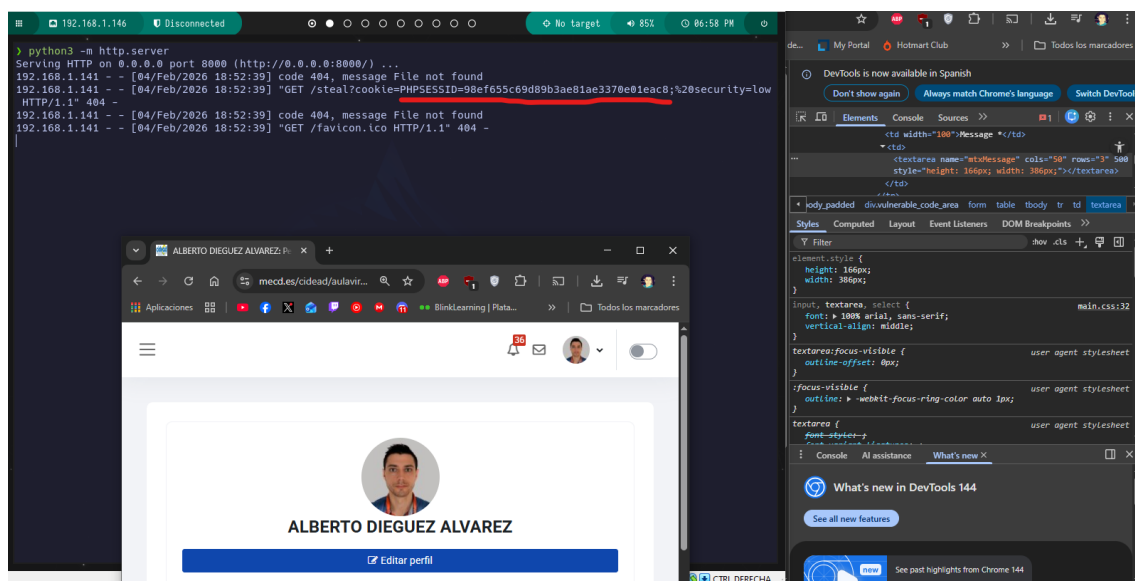
Primero levanto un http server en Python en el puerto 8000

```
python3 -m http.server
```

Modifico el textarea otra vez, y con el siguiente script, logro enviar la cookie de sesión a mi server y verla por consola.



Al ejecutarlo si miro mi servidor veo la cookie.



Apartado 2: Preguntas sobre el ejercicio

1. Rellena la siguiente tabla comparando la vulnerabilidad XSS (Cross-Site Scripting) con la vulnerabilidad SQL Injection

Cuestión	XSS	SQL Injection
Definición	Inyección de código malicioso que se ejecuta en el navegador del usuario.	Inyección de sentencias SQL maliciosas en consultas a la BBDD.
Tipos	Reflejado, almacenado y basado en DOM.	In-band, Blind y Out-of-band
Objetivo principal	Robar sesiones, credenciales o ejecutar acciones pasándose por el usuario.	Acceder, modificar o borrar datos de la BBDD.
Lenguaje/Ámbito Afectado	HTML, JavaScript	SQL

2. Rellena la siguiente tabla comparando la vulnerabilidad XSS (Cross-Site Scripting) con la vulnerabilidad SQL Injection

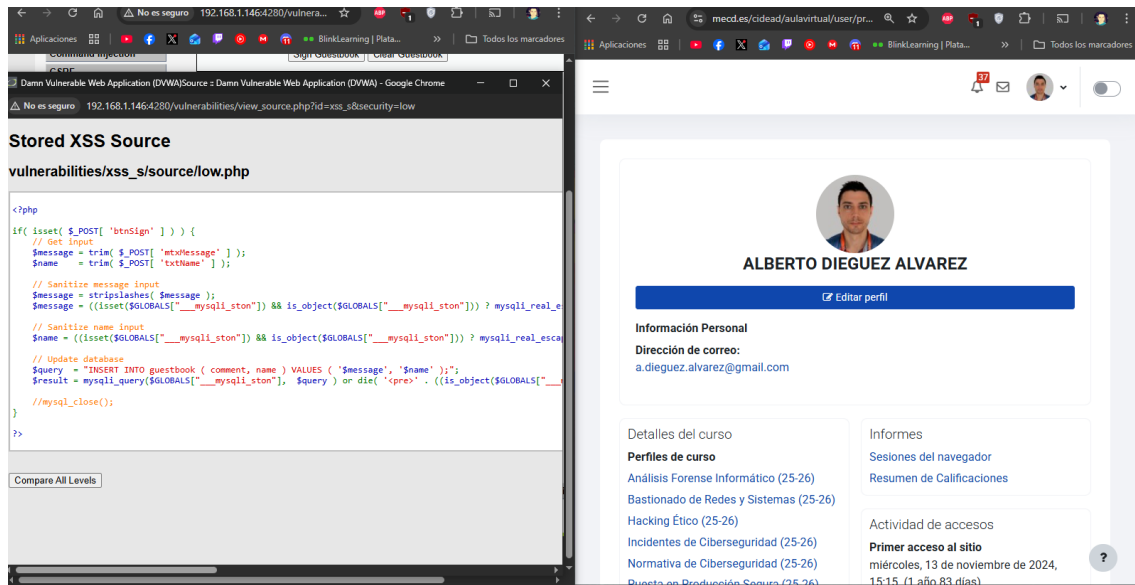
Cuestión	XSS	SQL Injection
Impacto	Robo de sesiones, ejecutar acciones maliciosas, suplantación de usuarios	Robo, modificación o borrado de datos
Ejemplos CVE (uno para cada tipo)	CVE-2024-29184 Permite inyectar código JavaScript en la firma del usuario y luego se ejecuta en el navegador del administrador	CVE-2024-8465 En la aplicación Job Portal (PHPGurukul) el parámetro user_id permite al atacante enviar consultas
Métodos de inyección	Entrada de texto en formularios, URLs, comentarios...	Campos de login, formularios, URL sin validación
Mitigaciones	Validar y escapar entradas, codificar salida, usar Content Security Policy	Consultas preparadas, validar entradas, privilegios mínimos.

3. Rellena la siguiente tabla sobre el ejemplo XSS (Cross-Site Scripting) del apartado 1

Cuestión	Respuesta
¿Este XSS es temporal o permanente? Justifica la respuesta	Es permanente, porque el script se guarda en el libro de firmas y se ejecuta cada vez que se visita la página
Indica las mejoras que se podrían hacer (indicando si se aplican en el lado del cliente o en el lado del servidor), comentadas durante el curso. Justifica la respuesta.	Servidor: • Validar la entrada (eliminar etiquetas) • Escapar la salida Justificación: El servidor controla lo que se guarda y se envía. Cliente: • Validaciones JavaScript en formularios Justificación: Evita errores accidentales del usuarios, pero no es muy eficaz.
¿Se podría evitar este XSS sólo en el lado del navegador? Justifica la respuesta	No ya que el atacante puede saltarse el navegador y enviar la petición directa al servidor.

4. Haz una comparativa del código php del ejemplo XSS del apartado 1 indicando los controles de seguridad implementados en el modo **Low** y en el modo **Impossible**

Reviso en Low:



```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );
    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) &&
is_object($GLOBALS["__mysqli_ston"]))) ?
mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call! This
code does not work.", E_USER_ERROR)) ? "" : "");
    // Sanitize name input
    $name = ((isset($GLOBALS["__mysqli_ston"]) &&
is_object($GLOBALS["__mysqli_ston"]))) ?
mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call! This
code does not work.", E_USER_ERROR)) ? "" : "");
    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message',
'$name' );";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die(
'<pre>' . ((is_object($GLOBALS["__mysqli_ston"]))) ?
mysqli_error($GLOBALS["__mysqli_ston"]) : (($__mysqli_res =
mysqli_connect_error()) ? $__mysqli_res : false)) . '</pre>' );
    //mysqli_close();
}
?>
```

Reviso en imposible:

Stored XSS Source

vulnerabilities/xss_s/source/impossible.php

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message) : htmlspecialchars( $message ));

    // Sanitize name input
    $name = stripslashes( $name );
    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name) : htmlspecialchars( $name ));

    // Update database
    $data = $db->prepare( 'INSERT INTO guestbook ( comment, name ) VALUES ( :message, :name )' );
    $data->bindParam( ':message', $message, PDO::PARAM_STR );
    $data->bindParam( ':name', $name, PDO::PARAM_STR );
    $data->execute();
}

// Generate Anti-CSRF token
generateSessionToken();
?>
```

Compare All Levels

ALBERTO DIEGUEZ ALVAREZ

[\[?\] Editar perfil](#)

Información Personal
Dirección de correo:
a.dieguez.alvarez@gmail.com

Detalles del curso
Perfiles de curso
Análisis Forense Informático (25-26)
Bastionado de Redes y Sistemas (25-26)
Hacking Ético (25-26)
Incidentes de Ciberseguridad (25-26)
Normativa de Ciberseguridad (25-26)
Quinta de Reducción de Riesgos (25-26)

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) &&
is_object($GLOBALS["__mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call! This code does not
work.", E_USER_ERROR)) ? "" : ""));
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = stripslashes( $name );
    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call! This code
does not work.", E_USER_ERROR)) ? "" : ""));
    $name = htmlspecialchars( $name );

    // Update database
    $data = $db->prepare( 'INSERT INTO guestbook ( comment, name ) VALUES ( :message,
:name );' );
    $data->bindParam( ':message', $message, PDO::PARAM_STR );
    $data->bindParam( ':name', $name, PDO::PARAM_STR );
    $data->execute();
}

// Generate Anti-CSRF token
generateSessionToken();
?>
```

Comparándolos realizo la siguiente tabla.

Aspecto	LOW	IMPOSSIBLE
Validación de entrada	No valida contenido HTML ni JS	Valida parcialmente
Escape de salida	No hay escape de caracteres	Realiza un escape de caracteres con \$message = htmlspecialchars(\$message);
Protección CSRF	No tiene	Tiene token anti CSRF
Acceso a BBDD	No prepara las sentencias y concatena SQL	Prepara las consultas con \$db->prepare

Notas:

- <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>
- <https://nvd.nist.gov/vuln/detail/CVE-2024-29184>
- <https://nvd.nist.gov/vuln/detail/CVE-2024-8465>